

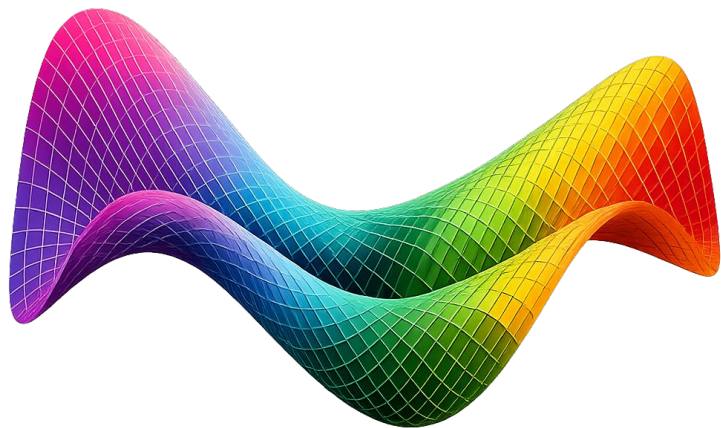
# Superfícies Paramétricas

---

# Do domínio bidimensional ao espaço tridimensional

Nesta aula, expandiremos os conceitos de curvas paramétricas para aplicá-los à modelagem de **superfícies tridimensionais**.

Entenderemos a matemática por trás da geração de geometria e suas vantagens diretas em relação aos modelos estáticos na Computação Gráfica.



# A transição de dimensões

Para modelar uma curva paramétrica, vimos que um único parâmetro **t** é suficiente para descrever uma trajetória unidimensional no espaço.

No entanto, formas tridimensionais contínuas, como o capô de um carro ou as ondulações de um terreno, possuem comprimento e largura.

Para descrever matematicamente essa área, precisamos de uma transição:

- O parâmetro único dá lugar a **dois parâmetros independentes**.
- Por convenção na literatura gráfica, chamamos esses parâmetros de **u** e **v**.

# O domínio paramétrico

A geração de uma superfície tridimensional parte de uma base plana bidimensional. Você pode visualizar essa base como um tecido perfeitamente quadrado.

A variável **u** define a posição horizontal neste plano, enquanto a variável **v** define a posição vertical. Assim como ocorria nas curvas, o domínio paramétrico padrão estabelece que ambas as variáveis variem de 0 a 1 ( $0 \leq u \leq 1$  e  $0 \leq v \leq 1$ ).

O **cruzamento contínuo dessas coordenadas** formará uma **malha** que será calculada e deformada para existir no mundo 3D.

# Representação matemática

A superfície paramétrica é representada por uma função vetorial de duas variáveis, responsável por mapear o plano de origem para as coordenadas reais do espaço 3D.

$$S(u, v) = (x(u,v), y(u,v), z(u,v))$$

Composição da equação:

- **Entrada:**  $(u, v)$  representam as coordenadas de um ponto específico no nosso plano paramétrico de base.
- **Processamento:**  $x(u,v)$ ,  $y(u,v)$  e  $z(u,v)$  são funções matemáticas separadas que recebem os parâmetros e calculam a nova posição geométrica.
- **Saída:**  $S(u, v)$  é o vértice final gerado, posicionado com exatidão nos eixos X, Y e Z da cena.

# Vantagem prática 1 - Resolução e LOD

Modelos tridimensionais estáticos (como arquivos OBJ ou FBX) possuem um número de polígonos predefinido. Se a câmera virtual se aproxima muito, a geometria revela seus limites, os triângulos ficam visíveis e o objeto parece facetado.

Em contrapartida, as superfícies paramétricas possuem resolução matemática infinita. Como os vértices são calculados dinamicamente, podemos integrar sistemas de Nível de Detalhe (LOD) alterando apenas o "passo" do laço de repetição:

- **Câmera próxima:** Avançamos **u** e **v** de 0.01 em 0.01, gerando uma malha bastante densa e suave.
- **Câmera distante:** Avançamos os parâmetros de 0.2 em 0.2, gerando menos polígonos e poupando a placa de vídeo.

## Vantagem prática 2 - Armazenamento e geração

A formulação paramétrica também otimiza drasticamente a **criação e o armazenamento de recursos geométricos**.

Objetos de base, como **esferas**, **cilindros** e **toros**, podem ser construídos de **forma procedural** puramente via cálculo na GPU, sem a necessidade de importar arquivos de modelagem externa.

Além disso, em vez de enviar milhares de coordenadas de vértices, normais e mapeamentos de textura para a memória de vídeo, a engine gráfica necessita armazenar apenas a função matemática e uma matriz reduzida de pontos de controle para gerar a mesma superfície.

# Primitivas Paramétricas Comuns

## A construção visual a partir da matemática

Antes de explorarmos superfícies complexas e deformações arbitrárias, precisamos entender como as **formas clássicas** da geometria 3D são geradas do zero.

Em motores gráficos (como o OpenGL, que muitos de vocês já utilizam), primitivas como **esferas** e **cilindros** não precisam ser carregadas de um arquivo externo. Elas possuem um "algoritmo matemático" nativo. O segredo dessas primitivas é o uso extensivo da **trigonometria**: combinamos senos e cossenos para criar revoluções no espaço.

Para todas as formas a seguir, nossos parâmetros **u** e **v** atuarão como ângulos ou posições ao longo de um eixo.

# O Algoritmo da Esfera

## Coordenadas Esféricas: Latitude e Longitude

A lógica matemática para desenhar uma esfera baseia-se em dois ângulos que varrem o espaço tridimensional, semelhante ao sistema de coordenadas do planeta Terra:

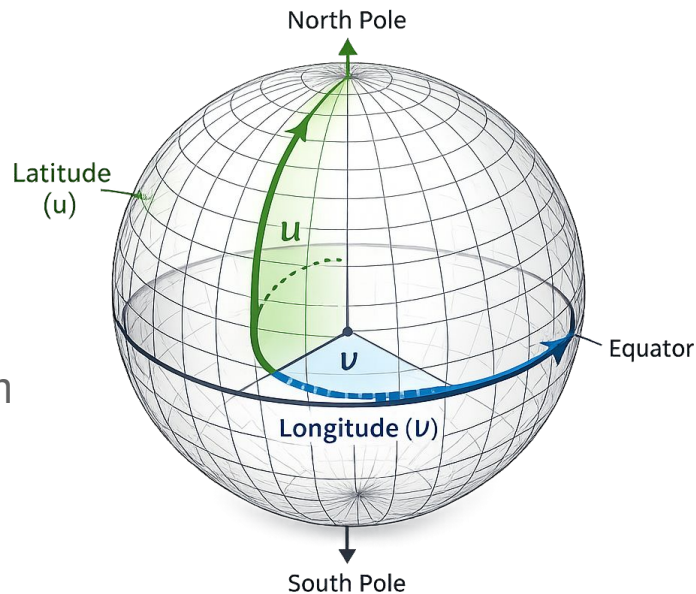
- O parâmetro **u** atua como a **latitude**, descendo do polo norte ao polo sul (variando de 0 a  $\pi$ ).
- O parâmetro **v** atua como a **longitude**, dando uma volta completa ao redor do equador (variando de 0 a  $2\pi$ ).

# O Algoritmo da Esfera

Equações da Esfera:

- $x = r * \sin(u) * \cos(v)$
- $y = r * \sin(u) * \sin(v)$
- $z = r * \cos(u)$

Ao calcularmos essas três equações para diversas combinações de **u** e **v**, os vértices gerados formam a casca de uma esfera perfeita de raio **r**.



# O Algoritmo do Cilindro

## Superfície de Revolução por Extrusão

A geração de um cilindro é um dos algoritmos mais simples de compreender. Nós combinamos o desenho de um círculo 2D com um deslocamento linear no terceiro eixo (extrusão).

### A Lógica do Algoritmo:

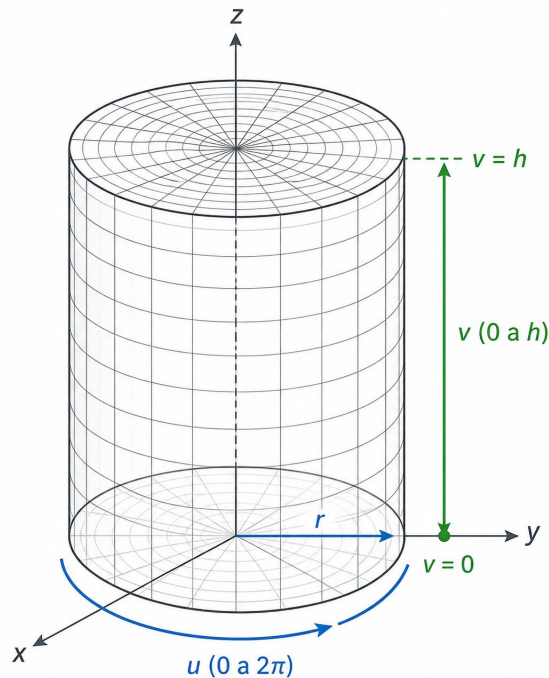
- O parâmetro **u** é responsável por desenhar a **base circular**. Ele varia de 0 a  $2\pi$  e é passado para as funções de seno e cosseno.
- O parâmetro **v** não é um ângulo, mas sim a **altura linear**. Ele "puxa" esse círculo ao longo do eixo Z.

# O Algoritmo do Cilindro

## Equações do Cilindro:

- $x = r * \cos(u)$
- $y = r * \sin(u)$
- $z = v$

Neste caso, enquanto **u** varre os 360 graus para desenhar os anéis, o **v** sobe gradativamente da base (0) até a altura máxima desejada, empilhando esses anéis para formar o tubo do cilindro.



# O Algoritmo do Cone

## Superfície de Revolução com Afunilamento

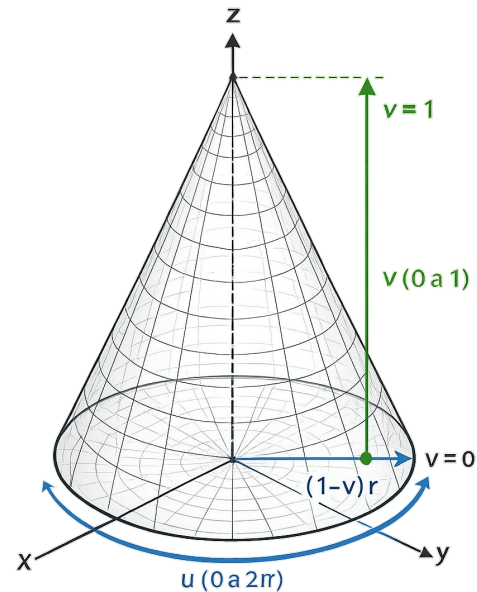
O cone utiliza a exata mesma lógica base do cilindro (um círculo extrudado ao longo de um eixo), mas com uma modificação matemática no raio: ele sofre um afunilamento proporcional à altura.

### A Lógica do Algoritmo:

- O parâmetro **u** continua girando 360 graus para fazer o círculo. O parâmetro **v** continua representando a altura (variando de 0 a 1).
- A diferença é que multiplicamos o raio original por  $(1 - v)$ . Assim, quando a altura **v** é 0 (base), o multiplicador é 1 (raio total). Quando a altura **v** chega a 1 (topo), o multiplicador zera, unindo todos os vértices em um único ponto.

### Equações do Cone:

- $x = (1 - v) * r * \cos(u)$
- $y = (1 - v) * r * \sin(u)$
- $z = v$



# O Algoritmo do Toróide/Torus (Donut)

## Círculos orbitando círculos

O toro é o exemplo definitivo do poder da parametrização dupla. A lógica matemática aqui envolve dois raios distintos: o raio do tubo do "donut" ( $r$ ) e o raio principal do centro até o meio do tubo ( $R$ ).

## A Lógica do Algoritmo:

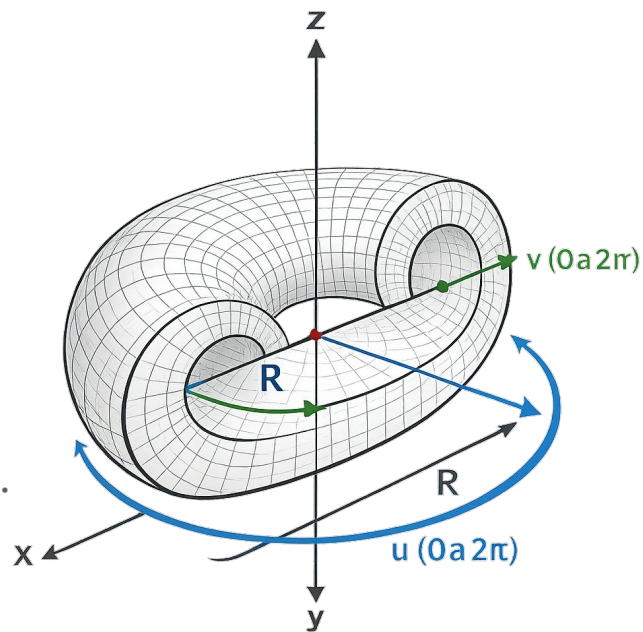
- O parâmetro **v** é usado para desenhar o pequeno círculo que forma a espessura do tubo (usando o raio ' $r$ ').
- O parâmetro **u** pega esse círculo pronto e o rotaciona pelo espaço ao redor de um eixo central, criando um anel contínuo (usando o raio ' $R$ ').

# O Algoritmo do Toróide/Torus (Donut)

## Equações do Toro:

- $x = (R + r * \cos(v)) * \cos(u)$
- $y = (R + r * \cos(v)) * \sin(u)$
- $z = r * \sin(v)$

A variável **z** depende apenas de **v** porque ela representa a altura local do próprio tubo, independentemente de onde ele está na órbita global.



# Discretização - Da matemática para a GPU

## O processo de Tessellation na CPU

Até agora, tratamos as superfícies paramétricas como funções matemáticas contínuas perfeitas. No entanto, o hardware gráfico (a GPU) renderiza o mundo através de **primitivas discretas**, predominantemente **triângulos**.

Para renderizar nossas equações no OpenGL, precisamos realizar um processo chamado discretização (ou tesselação na CPU). Isso significa **transformar a matemática abstrata do nosso domínio contínuo em uma lista finita e ordenada de dados reais**: um conjunto de vértices e um conjunto de índices que formam uma **malha triangular**.

# Amostragem do grid

## Dividindo o domínio contínuo em passos discretos

O primeiro passo da discretização é **fatiar** o nosso domínio paramétrico (o plano bidimensional **u,v**) em uma grade ("grid") de pontos. Definimos uma **resolução** que ditará quantos "passos" (steps) daremos do valor 0 até o valor 1.

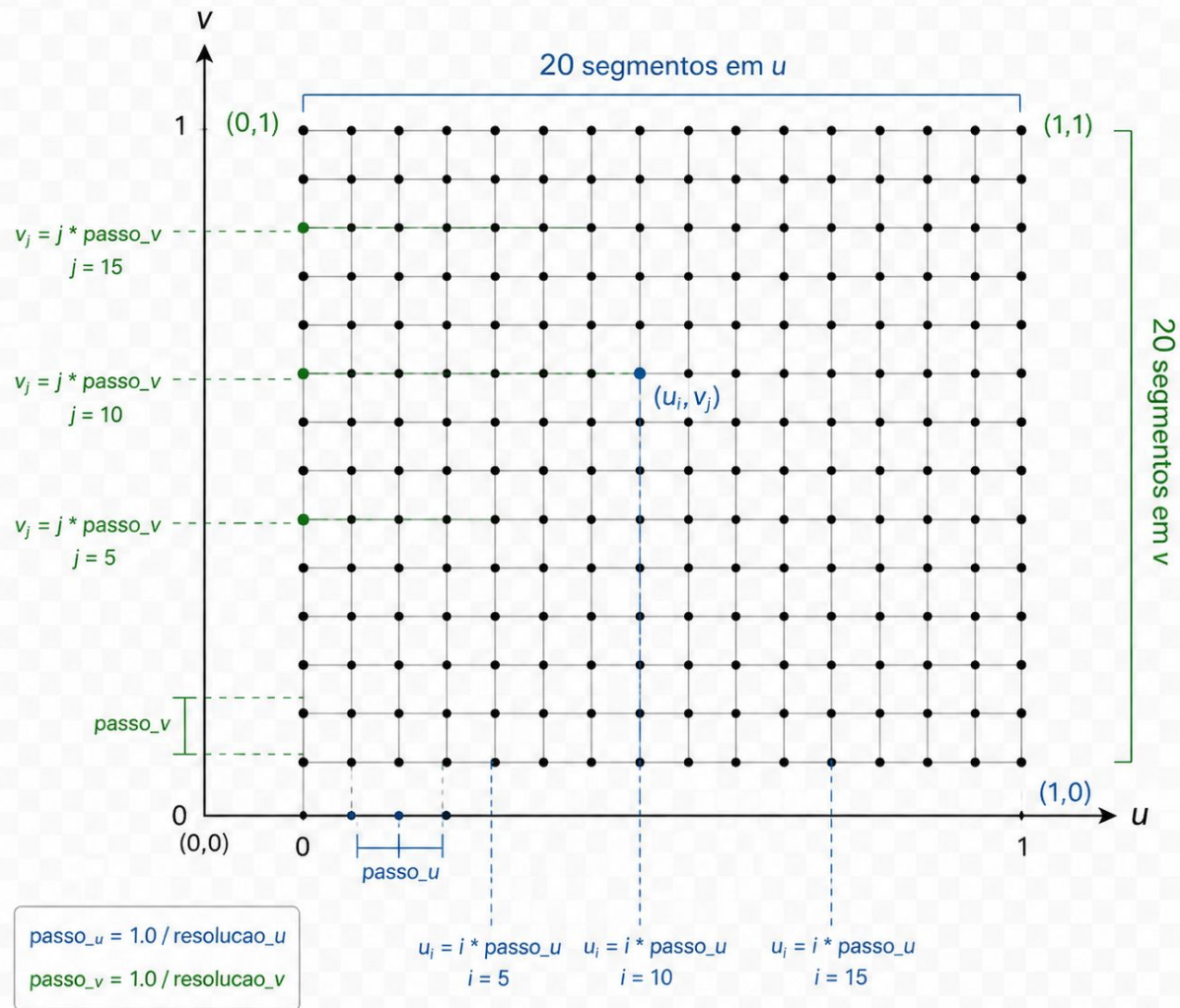
### A lógica do passo:

Se escolhermos uma resolução de 20x20, significa que o **plano será dividido em 20 segmentos no eixo u e 20 segmentos no eixo v**.

O **tamanho de cada passo** será calculado simplesmente dividindo o espaço total pela resolução:

- $\text{passo\_u} = 1.0 / \text{resolucao\_u}$
- $\text{passo\_v} = 1.0 / \text{resolucao\_v}$

Ao iterar de 0 a 20, multiplicamos o índice atual pelo tamanho do passo para descobrir a coordenada exata (u, v) que vamos amostrar.



# Geração de vértices

## Avaliando a função matemática

Com os nossos passos definidos, a implementação ocorre através de um laço de repetição duplo (um loop aninhado). Para cada linha 'v' da nossa grade paramétrica, nós percorremos todas as colunas 'u'.

Estrutura lógica do algoritmo:

Para cada valor de **v** (de 0 até **resolucao\_v**):

Para cada valor de **u** (de 0 até **resolucao\_u**):

1. Calcular o **u\_atual** e **v\_atual**
2. Passar (**u\_atual**, **v\_atual**) para a nossa equação  $S(u,v)$
3. Obter as coordenadas (**x**, **y**, **z**) resultantes
4. Guardar esse vértice em um array na memória principal (RAM)

O resultado desse processo é uma lista linear de pontos (uma "nuvem de pontos") que já possui o formato exato da nossa superfície 3D final, como uma esfera ou um cilindro.

# Atributos da superfície geométrica

## Os dados essenciais para o pipeline de renderização

A renderização de materiais e cálculos de iluminação no OpenGL exigem mais do que apenas a posição (x, y, z) dos vértices. A placa de vídeo (através dos Shaders) precisa receber **coordenadas de textura e vetores direcionais** para calcular como a luz interage com a geometria.

Na modelagem poligonal estática, esses atributos precisam ser gerados por algoritmos externos. Na geração paramétrica, os atributos de renderização são extraídos **nativamente** das variáveis que já compõem a função da superfície.

# Coordenadas de textura (UV)

## O mapeamento direto no código

O mapeamento de texturas na computação gráfica utiliza um sistema de coordenadas chamado UV, onde os eixos variam de 0.0 a 1.0.

Os parâmetros **u** e **v** que controlam o laço de repetição da nossa superfície operam **exatamente nessa mesma escala**. Consequentemente, as variáveis de iteração servem como as próprias **coordenadas de textura do modelo**, sem a necessidade de algoritmos de planificação (unwrap).

# A relação entre (u, v) e a trigonometria

## Traduzindo os parâmetros para o código

Na teoria, vimos que uma superfície é gerada varrendo os parâmetros **u** e **v** de 0 a 1. Na matemática esférica implementada no código, nós simplesmente mapeamos esse **u** e **v** para os ângulos de **latitude** e **longitude**.

Para ilustrar o cálculo analítico das derivadas no próximo slide, usaremos a **Esfera** como exemplo. Na literatura de geometria, você verá as variáveis **phi** ( $\varphi$ ) e **theta** ( $\theta$ ):

- O ângulo **phi** ( $\varphi$ ) é o nosso parâmetro **u** mapeado para a **latitude** (variando de 0 até  $\pi$ ). Ele desce do topo ao fundo da esfera.
- O ângulo **theta** ( $\theta$ ) é o nosso parâmetro **v** mapeado para a **longitude** (variando de 0 até  $2\pi$ ). Ele dá a volta completa ao redor da esfera.

É exatamente o mesmo conceito de mapeamento de um plano 2D, apenas escalado para radianos!

# Vetores tangentes

## Direcionamento da superfície via derivadas parciais

Para implementar efeitos avançados de iluminação, como o Normal Mapping, o Shader necessita dos vetores Tangentes. Eles indicam a direção em que a geometria se desloca localmente ao longo da superfície.

Matematicamente, a tangente é obtida calculando a derivada parcial da função de posicionamento.

- A derivada da função em relação ao parâmetro **u** resulta no vetor Tangente U (**Tu**).
- A derivada da função em relação ao parâmetro **v** resulta no vetor Tangente V (**Tv**).

Exemplo do cálculo analítico para o eixo X da esfera:

- A equação original da posição:  $x = \text{radius} * \sin(\phi) * \cos(\theta)$
- A Tangente U (derivando o ângulo  $\phi$ ):  $t_x = \text{radius} * \cos(\phi) * \cos(\theta)$

Esse processo é repetido para y e z, gerando um vetor direcional 3D completo.

# Cálculo universal da Normal

## O produto vetorial na geometria procedimental

Em primitivas simples como a esfera centrada na origem, a **Normal** pode ser obtida dividindo a coordenada espacial pelo raio. Porém, em superfícies complexas ou terrenos procedurais deformados, esse método não funciona. O cálculo matemático universal da Normal exige o uso dos vetores tangentes.

A operação de Produto Vetorial (*Cross Product*) entre a **Tangente U** e a **Tangente V** resulta em um vetor estritamente ortogonal a ambos. Este vetor ortogonal é a **Normal da superfície naquele ponto específico**.

### A implementação com NumPy:

```
# Criação dos arrays com as componentes tangentes calculadas
Tu = np.array([tx, ty, tz])
Tv = np.array([vx, vy, vz])

# O produto vetorial resolve o vetor Normal
N = np.cross(Tu, Tv)

# Normalização do vetor (garantir que o comprimento seja 1)
N = N / np.linalg.norm(N)
```

# O Vertex Buffer consolidado

## O layout completo de dados para o OpenGL

Com os atributos calculados analiticamente a cada iteração da grade, estruturamos a lista de vértices para conter o pacote completo de informações exigido pelo hardware gráfico.

A estrutura de dados a ser empacotada:

```
# Este bloco substitui o extend() dos passos anteriores
vertices.extend([
    x, y, z,          # Posição 3D (3 floats)
    N[0], N[1], N[2], # Vetor Normal (3 floats)
    u, v,            # Coordenadas UV (2 floats)
    tx, ty, tz       # Vetor Tangente U (3 floats)
])
```

- Cada vértice agora consome 11 floats na memória.
- O atributo **stride** no momento de configuração do OpenGL será atualizado para `11 * ctypes.sizeof(ctypes.c_float)`.

O **Vertex Array Object (VAO)** resultante possui agora todos os elementos estruturais para viabilizar cálculos complexos de sombreamento (Shading) e simulações de materiais fisicamente corretos (PBR).

# Geração de índices

## A triangulação da malha (Quad Grid)

O OpenGL não saberá o que fazer apenas com uma lista de vértices soltos. Precisamos informar à GPU como conectar esses pontos para formar faces sólidas. Como nossa grade gerou pequenos quadrados ("quads") a partir do cruzamento das linhas **u** e **v**, o objetivo agora é **dividir cada um desses quadrados em dois triângulos**.

Para fazer isso, não duplicamos os vértices na memória. Em vez disso, criamos um *Element Buffer Object* (EBO), que é um array contendo números inteiros. Cada número aponta para a posição de um vértice no array original.

A conexão ocorre **agrupando os índices de 3 em 3**. Para cada quadrado da grade paramétrica, precisamos gerar 6 índices, compondo 2 triângulos vizinhos.

# O algoritmo de triangulação

## Calculando a topologia da superfície

Para conectar os vértices corretamente via código, precisamos de uma fórmula que traduza a posição bidimensional (linha e coluna) do nosso grid paramétrico para a posição linear (o índice) no array do EBO.

Considerando que "largura" é o número total de vértices em uma linha (`resolucao_v + 1`):

- O vértice atual superior esquerdo =  $(\text{linha} * \text{largura}) + \text{coluna}$
- O vértice atual superior direito =  $(\text{linha} * \text{largura}) + \text{coluna} + 1$
- O vértice inferior esquerdo =  $((\text{linha} + 1) * \text{largura}) + \text{coluna}$
- O vértice inferior direito =  $((\text{linha} + 1) * \text{largura}) + \text{coluna} + 1$

Com esses quatro índices calculados em mãos a cada iteração, enviamos duas trincas para o OpenGL:

- **Triângulo 1:** Superior Esquerdo, Inferior Esquerdo, Superior Direito
- **Triângulo 2:** Superior Direito, Inferior Esquerdo, Inferior Direito

# O pipeline final

## Da CPU para a Placa de Vídeo

Resumo do fluxo de implementação para carregar nossa primitiva matemática na engine gráfica:

- **Setup:** Definimos as equações  $S(u,v)$  da primitiva e a resolução desejada.
- **Avaliação:** O loop duplo gera as coordenadas  $(x, y, z)$ , popularizando o *Vertex Buffer Object* (VBO).
- **Topologia:** Um segundo loop gera a ordem dos triângulos, preenchendo o *Element Buffer Object* (EBO).
- **Draw Call:** Instruímos o OpenGL a renderizar os buffers utilizando a instrução `GL_TRIANGLES`.

Ao alterar a resolução no início do processo, o mesmo algoritmo gerará uma esfera com 100 triângulos ou 10.000 triângulos, demonstrando o poder do Nível de Detalhe (LOD) dinâmico.

# A Arquitetura do Gerador Geométrico

## O Padrão de Implementação (Template)

Para gerar qualquer superfície paramétrica, não precisamos reescrever a configuração do OpenGL. Vamos estabelecer um **Algoritmo Base**.

Este algoritmo fará todo o trabalho pesado: os laços de repetição, o empacotamento dos 11 atributos do vértice e a triangulação da malha.

A única tarefa do programador será preencher um "Bloco Matemático" específico com três informações fundamentais da superfície desejada:

- **Posição ( $x, y, z$ ):** A equação paramétrica base.
- **Tangente U ( $t_x, t_y, t_z$ ):** A derivada parcial em relação a **u**.
- **Tangente V ( $v_x, v_y, v_z$ ):** A derivada parcial em relação a **v**.

# O Algoritmo Base

## Passo 0: Adicionando as importações

```
import numpy as np
import math
import ctypes
from OpenGL.GL import *
```

# O Algoritmo Base

## Passo 1: O Laço Gerador e a abstração UV

```
def inicializaSuperficie():  
    global Vao_geometria, Num_indices_geometria  
  
    resolucao_u = 30 # Passos longitudinais  
    resolucao_v = 30 # Passos transversais  
    vertices = []  
    passo_u = 1/resolucao_u  
    passo_v = 1/resolucao_v  
    largura = resolucao_v + 1  
  
    # O loop principal da geometria  
    for i in range(resolucao_u + 1):  
        u = i * passo_u  
        for j in range(resolucao_v + 1):  
            v = j * passo_v  
  
            # --- INÍCIO DO BLOCO MATEMÁTICO DA FORMA ---  
            # É AQUI que definiremos x, y, z, tx, ty, tz, vx, vy, vz  
            # de acordo com o objeto que queremos gerar.  
            # --- FIM DO BLOCO MATEMÁTICO DA FORMA ---
```

# O Algoritmo Base

## Passo 2: Cálculo Universal da Normal e Empacotamento

Logo após o "Bloco Matemático", usamos as tangentes fornecidas para calcular a Normal de forma procedural, fechando os atributos do vértice:

```
# Produto vetorial das tangentes para obter a Normal
Tu = np.array([tx, ty, tz])
Tv = np.array([vx, vy, vz])
N = np.cross(Tu, Tv)

# Normalização (evitando divisão por zero)
norma = np.linalg.norm(N)
if norma != 0:
    N = N / norma

# Empacotamento dos 11 floats do vértice atual
vertices.extend([
    x, y, z,                # Offset 0: Posição
    N[0], N[1], N[2],       # Offset 12: Normal procedural
    u, v,                  # Offset 24: Coordenadas UV
    tx, ty, tz             # Offset 32: Tangente U
])
```

# O Algoritmo Base

## Passo 3: Topologia Invariante

A malha gerada por parâmetros **u** e **v** sempre forma um grid retangular perfeito no domínio 2D. Portanto, o algoritmo de geração de índices (triangulação) **é exatamente o mesmo para qualquer primitiva.**

```
indices = []
for i in range(resolucao_u):
    for j in range(resolucao_v):
        # Índices lineares da célula atual
        sup_esq = i * largura + j
        sup_dir = sup_esq + 1
        inf_esq = (i + 1) * largura + j
        inf_dir = inf_esq + 1

        # Geração dos dois triângulos (sentido anti-horário)
        indices.extend([sup_esq, inf_esq, sup_dir])
        indices.extend([sup_dir, inf_esq, inf_dir])

Num_indices_geometria = len(indices)
```

# O Algoritmo Base

## Passo 4: Configuração dos Buffers (Boilerplate)

O empacotamento em arrays do NumPy e o envio para o Vertex Array Object (VAO) também se tornam código reaproveitável.

```
vertices_data = np.array(vertices, dtype=np.float32)
indices_data = np.array(indices, dtype=np.uint32)

Vao_geometria = glGenVertexArrays(1)
vbo = glGenBuffers(1)
ebo = glGenBuffers(1)

glBindVertexArray(Vao_geometria)

glBindBuffer(GL_ARRAY_BUFFER, vbo)
glBufferData(GL_ARRAY_BUFFER, vertices_data.nbytes, vertices_data, GL_STATIC_DRAW)

glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, ebo)
glBufferData(GL_ELEMENT_ARRAY_BUFFER, indices_data.nbytes, indices_data, GL_STATIC_DRAW)
```

# O Algoritmo Base

## Passo 4: Configuração dos Buffers (Boilerplate)

```
# Definição do layout de memória. Cada vértice tem 11 floats (44 bytes)
float_size = ctypes.sizeof(ctypes.c_float)
stride = 11 * float_size

# Atributo 0: Posição (3 floats)
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, stride, ctypes.c_void_p(0))
glEnableVertexAttribArray(0)

# Atributo 1: Normal (3 floats, começa após os 3 primeiros floats)
glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, stride, ctypes.c_void_p(3 * float_size))
glEnableVertexAttribArray(1)

# Atributo 2: Coordenadas UV (2 floats, começa após 6 floats)
glVertexAttribPointer(2, 2, GL_FLOAT, GL_FALSE, stride, ctypes.c_void_p(6 * float_size))
glEnableVertexAttribArray(2)

# Atributo 3: Tangente U (3 floats, começa após 8 floats)
glVertexAttribPointer(3, 3, GL_FLOAT, GL_FALSE, stride, ctypes.c_void_p(8 * float_size))
glEnableVertexAttribArray(3)

glBindVertexArray(0)
```

# O Algoritmo Base

## Passo 5: Renderização indexada

Com a função de inicialização pronta, o desenho no loop de renderização passa a utilizar o comando indexado **glDrawElements**, que é muito mais eficiente.

```
glBindVertexArray(Vao_geometria)  
glDrawElements(GL_TRIANGLES, Num_indices_geometria, GL_UNSIGNED_INT, None)
```

Diferente do **glDrawArrays**, o comando **glDrawElements** instrui o hardware a buscar a ordem dos vértices diretamente no buffer de índices que criamos (EBO).

Isso permite que a placa de vídeo processe cada vértice único apenas uma vez, mesmo que ele seja compartilhado por vários triângulos, aumentando o desempenho global da aplicação.

# A Versatilidade da Modelagem Dinâmica

A arquitetura que acabamos de construir (laços de repetição, geração de índices e buffers) é uma **infraestrutura invariante**. Isso significa que, para criar objetos diferentes, você não precisa reescrever a lógica do motor gráfico ou da GPU.

O segredo da geração procedural é apenas substituir as equações matemáticas específicas no interior do laço.

Nas páginas a seguir, aplicaremos essa lógica para "plugar" a matemática de uma **Esfera** e de um **Terreno Procedural** dentro deste mesmo gerador

# Exemplo de Aplicação

## Injetando a Esfera no Algoritmo Base

Para gerar uma esfera perfeita com suporte a iluminação avançada, substituímos o **Bloco Matemático** pela equação paramétrica da casca esférica. Primeiro, convertemos o **u** e **v** abstratos para os ângulos reais e calculamos a posição 3D:

```
# --- INÍCIO DO BLOCO MATEMÁTICO DA FORMA ---  
# Exemplo: ESFERA  
radius = 0.5  
  
# 1. Mapeamento dos ângulos  
phi = u * math.pi  
theta = v * - 2.0 * math.pi  
  
# 2. Posição (S(u,v))  
x = radius * math.sin(phi) * math.cos(theta)  
y = radius * math.cos(phi)  
z = radius * math.sin(phi) * math.sin(theta)
```

# Exemplo de Aplicação

## As derivadas da esfera

Na sequência do bloco, implementamos as tangentes derivadas da posição. O algoritmo base usará isso logo na linha seguinte para calcular as normais e enviar para a GPU.

```
# 3. Tangentes (Derivadas parciais)
dphi_du = math.pi
dtheta_dv = -2.0 * math.pi

# Derivando S em relação a 'u' (Tangente U via phi)
tx = radius * math.cos(phi) * math.cos(theta) * dphi_du
ty = -radius * math.sin(phi) * dphi_du
tz = radius * math.cos(phi) * math.sin(theta) * dphi_du

# Derivando S em relação a 'v' (Tangente V via theta)
vx = -radius * math.sin(phi) * math.sin(theta) * dtheta_dv
vy = 0.0
vz = radius * math.sin(phi) * math.cos(theta) * dtheta_dv
# --- FIM DO BLOCO MATEMÁTICO DA FORMA ---
```

# Exemplo de Aplicação 2

## O Terreno Procedural (Superfície Senoidal)

Para criar um terreno ondulado (como dunas ou água), não alteramos nada na arquitetura do OpenGL. Apenas substituímos o Bloco Matemático. O domínio  $(u,v)$  mapeia diretamente para os eixos X e Z (o chão), e a altura Y é dada por uma função trigonométrica:

```
# --- INÍCIO DO BLOCO MATEMÁTICO DA FORMA ---  
# Exemplo: TERRENO ONDULADO  
freq = 10.0      # Frequência das montanhas  
amplitude = 0.2  # Altura das montanhas  
  
# 1. Posição (S(u,v))  
# Centralizamos X e Z entre -0.5 e 0.5  
x = u - 0.5  
z = 0.5 - v  
  
# A altura Y é uma onda composta  
y = amplitude * math.sin(u * freq) * math.cos(v * freq)
```

# Exemplo de Aplicação 2

## As derivadas do Terreno

Como os eixos X e Z são mapeamentos lineares diretos de **u** e **v**, as derivadas parciais são triviais ( $dx/du = 1$ ,  $dz/du = 0$ , etc.). O único cálculo real é a derivada da altura Y usando a regra da cadeia.

```
# 3. Tangentes (Derivadas parciais)
# Derivando a altura Y em relação a 'u' e 'v'
dy_du = amplitude * freq * math.cos(u * freq) * math.cos(v * freq)
dy_dv = -amplitude * freq * math.sin(u * freq) * math.sin(v * freq)

# Tangente U (derivadas de x, y, z em relação a u)
tx = 1.0
ty = dy_du
tz = 0.0

# Tangente V (derivadas de x, y, z em relação a v)
vx = 0.0
vy = dy_dv
vz = -1.0
# --- FIM DO BLOCO MATEMÁTICO DA FORMA ---
```